

УНИВЕРСИТЕТ ИТМО
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Лабораторная работа №4 по дисциплине «Вычислительная математика»

Вариант 13

Выполнил:

Снагин Станислав Максимович,
Р3215, 467525

Проверил:

Наумова Надежда Александровна

г. Санкт-Петербург, Россия
2026

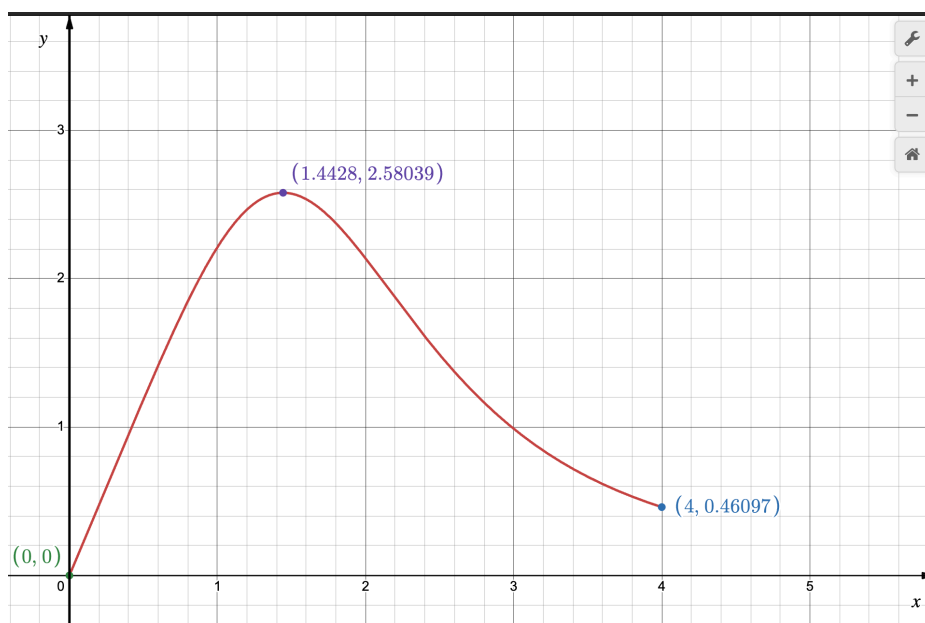
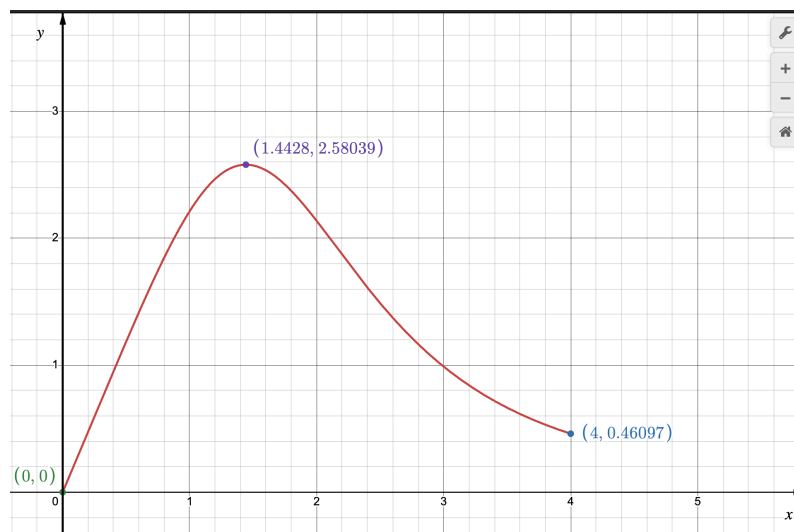
Содержание

1	Выполнение работы	1
1.1	Вычислительная часть	1
1.2	Программная часть	5
2	Заключение	14

1 Выполнение работы

1.1 Вычислительная часть

По варианту 13, имеем:



Вычисления, проводимые в [desmos](https://www.desmos.com/calculator/kqakqwmn6w)

<https://www.desmos.com/calculator/kqakqwmn6w>

i	0	1	2	3	4	5	6	7	8	9	10
x_i	0.0	0.4	0.8	1.2	1.6	2.0	2.4	2.8	3.2	3.6	4.0
y_i	0	0.951	1.849	2.468	2.537	2.138	1.611	1.166	0.842	0.617	0.461

Линейное приближение

Введем обозначения сумм

$$S_x, S_y, S_{xx}, S_{xy}$$

И посчитаем их:

$$S_x = \sum_{i=0}^{10} x_i = 0 + 0.4 + 0.8 + 1.2 + 1.6 + 2.0 + 2.4 + 2.8 + 3.2 + 3.6 + 4.0 = 22.0$$

$$S_y = \sum_{i=0}^{10} y_i = 0 + 0.951 + 1.849 + 2.468 + 2.537 + 2.138 + 1.611 + \\ + 1.166 + 0.842 + 0.617 + 0.461 = 14.640$$

$$S_{xx} = \sum_{i=0}^{10} x_i^2 = 0^2 + 0.4^2 + 0.8^2 + 1.2^2 + 1.6^2 + 2.0^2 + 2.4^2 + 2.8^2 + 3.2^2 + 3.6^2 + 4.0^2 = 61.60$$

$$S_{xy} = \sum_{i=0}^{10} x_i y_i = 0 \cdot 0 + 0.4 \cdot 0.951 + 0.8 \cdot 1.849 + 1.2 \cdot 2.468 + 1.6 \cdot 2.537 + \\ + 2.0 \cdot 2.138 + 2.4 \cdot 1.611 + 2.8 \cdot 1.166 + 3.2 \cdot 0.842 + 3.6 \cdot 0.617 + 4.0 \cdot 0.461 = 27.0472$$

По полученным данным построим систему уравнений:

$$\begin{cases} S_{xx}a + S_x b = S_{xy} \\ S_x a + n b = S_y, \end{cases} \Rightarrow \begin{cases} 61.6a + 22b = 24.3048 \\ 22a + 11b = 14.640 \end{cases}$$

Решим эту систему методом Крамера:

$$\Delta = \begin{vmatrix} 11 & 22 \\ 22 & 61.6 \end{vmatrix} = 11 \cdot 61.6 - 22 \cdot 22 = 677.6 - 484 = 193.6$$

$$\Delta_a = S_{xy} \cdot n - S_x \cdot S_y$$

$$\Delta_a = \begin{vmatrix} 14.640 & 22 \\ 27.0472 & 61.6 \end{vmatrix} = 14.640 \cdot 61.6 - 22 \cdot 27.0472 = 901.824 - 595.0384 = 306.7856$$

$$\Delta_b = S_{xx} \cdot S_y - S_x \cdot S_{xy}$$

$$\Delta_b = \begin{vmatrix} 11 & 14.640 \\ 22 & 27.0472 \end{vmatrix} = 11 \cdot 27.0472 - 14.640 \cdot 22 = 297.5192 - 322.08 = -24.5608$$

Т.е. коэффициенты:

$$a = \frac{\Delta_a}{\Delta} = \frac{306.7856}{193.6} \approx 1.5846$$

$$b = \frac{\Delta_b}{\Delta} = \frac{-24.5608}{193.6} \approx -0.1269$$

Итоговое линейное приближение

$$\boxed{\varphi(x) = 1.5846x - 0.1269}$$

Среднеквадратичное отклонение

i	0	1	2	3	4	5	6	7	8	9	10
x_i	0	0.4	0.8	1.2	1.6	2.0	2.4	2.8	3.2	3.6	4.0
y_i	0	0.951	1.849	2.468	2.537	2.138	1.611	1.166	0.842	0.617	0.461
$\varphi(x_i)$	-0.126	0.507	1.141	1.775	2.408	3.042	3.676	4.310	4.944	5.578	6.212
ε_i^2	0.016	0.197	0.501	0.480	0.017	0.817	4.264	9.885	16.826	24.612	33.074

$$\varepsilon_i = (\varphi(x_i) - y_i)$$

$$\text{Среднеквадратичное отклонение } \sigma = \sqrt{\frac{\sum \varphi(x_i) - (y_i)^2}{n}} = 2.871.$$

$$S = \sum_{i=0}^{10} \varepsilon_i^2 = 0.016 + 0.197 + 0.501 + 0.480 + 0.017 + 0.817 + 4.264 + 9.885 + 16.826 + 24.612 + 33.074 = 90.689$$

$$\sigma = \sqrt{\frac{S}{n}} = \sqrt{\frac{\sum_{i=0}^{10} (\varphi(x_i) - y_i)^2}{11}} = \sqrt{\frac{90.689}{11}} = \sqrt{8.244} \approx 2.871$$

Квадратичная аппроксимация

i	0	1	2	3	4	5	6	7	8	9	10
x_i	0.0	0.4	0.8	1.2	1.6	2.0	2.4	2.8	3.2	3.6	4.0
y_i	0	0.951	1.849	2.468	2.537	2.138	1.611	1.166	0.842	0.617	0.461

$$\varphi(x) = a + bx + cx^2$$

Вычислим суммы

$$S_x = \sum_{i=0}^{10} x_i = 22.0$$

$$S_{xx} = \sum_{i=0}^{10} x_i^2 = 61.6$$

$$S_{xxx} = \sum_{i=0}^{10} x_i^3 = 193.6$$

$$S_{xxxx} = \sum_{i=0}^{10} x_i^4 = 648.5248$$

$$S_y = \sum_{i=0}^{10} y_i = 14.640$$

$$S_{xy} = \sum_{i=0}^{10} x_i y_i = 27.0472$$

$$S_{xxy} = \sum_{i=0}^{10} x_i^2 y_i = 62.3514$$

По полученным данным построим систему уравнений:

$$\begin{cases} n \cdot a + S_x \cdot b + S_{xx} \cdot c = S_y \\ S_x \cdot a + S_{xx} \cdot b + S_{xxx} \cdot c = S_{xy} \\ S_{xx} \cdot a + S_{xxx} \cdot b + S_{xxxx} \cdot c = S_{xxy} \end{cases}$$

$$\Rightarrow \begin{cases} 11 \cdot a + 22 \cdot b + 61.6 \cdot c = 14.640 \\ 22 \cdot a + 61.6 \cdot b + 193.6 \cdot c = 27.0472 \\ 61.6 \cdot a + 193.6 \cdot b + 648.5248 \cdot c = 62.3514 \end{cases}$$

Из этой системы найдем коэффициенты а, b и c через метод Крамера:

$$\Delta = \begin{vmatrix} 11 & 22 & 61.6 \\ 22 & 61.6 & 193.6 \\ 61.6 & 193.6 & 648.5248 \end{vmatrix}$$

$$\Delta = 11 \cdot (61.6 \cdot 648.5248 - 193.6 \cdot 193.6) - 22 \cdot (22 \cdot 648.5248 - 193.6 \cdot 61.6) + 61.6 \cdot (22 \cdot 193.6 - 61.6 \cdot 61.6)$$

$$\Delta = 11 \cdot 2468.16768 - 22 \cdot 2341.7856 + 61.6 \cdot 464.64 = 4252.38528$$

$$\Delta_1 = \begin{vmatrix} 14.640 & 22 & 61.6 \\ 27.0472 & 61.6 & 193.6 \\ 62.3514 & 193.6 & 648.5248 \end{vmatrix}$$

$$\Delta_1 = 14.640 \cdot 2468.16768 - 22 \cdot 5469.54375 + 61.6 \cdot 1395.49168 = 1766.29976$$

$$\Delta_2 = \begin{vmatrix} 11 & 14.640 & 61.6 \\ 22 & 27.0472 & 193.6 \\ 61.6 & 62.3514 & 648.5248 \end{vmatrix}$$

$$\Delta_2 = 11 \cdot 5469.54375 - 14.640 \cdot 2341.7856 + 61.6 \cdot (-294.37672) = 7747.63415$$

$$\Delta_3 = \begin{vmatrix} 11 & 22 & 14.640 \\ 22 & 61.6 & 27.0472 \\ 61.6 & 193.6 & 62.3514 \end{vmatrix}$$

$$\Delta_3 = 11 \cdot (-1395.49168) - 22 \cdot (-294.37672) + 14.640 \cdot 464.64 = -2071.79104$$

$$a = \frac{\Delta_1}{\Delta} = \frac{1766.29976}{4252.38528} \approx 0.4154$$

$$b = \frac{\Delta_2}{\Delta} = \frac{7747.63415}{4252.38528} \approx 1.8220$$

$$c = \frac{\Delta_3}{\Delta} = \frac{-2071.79104}{4252.38528} \approx -0.4872$$

Квадратичное приближение

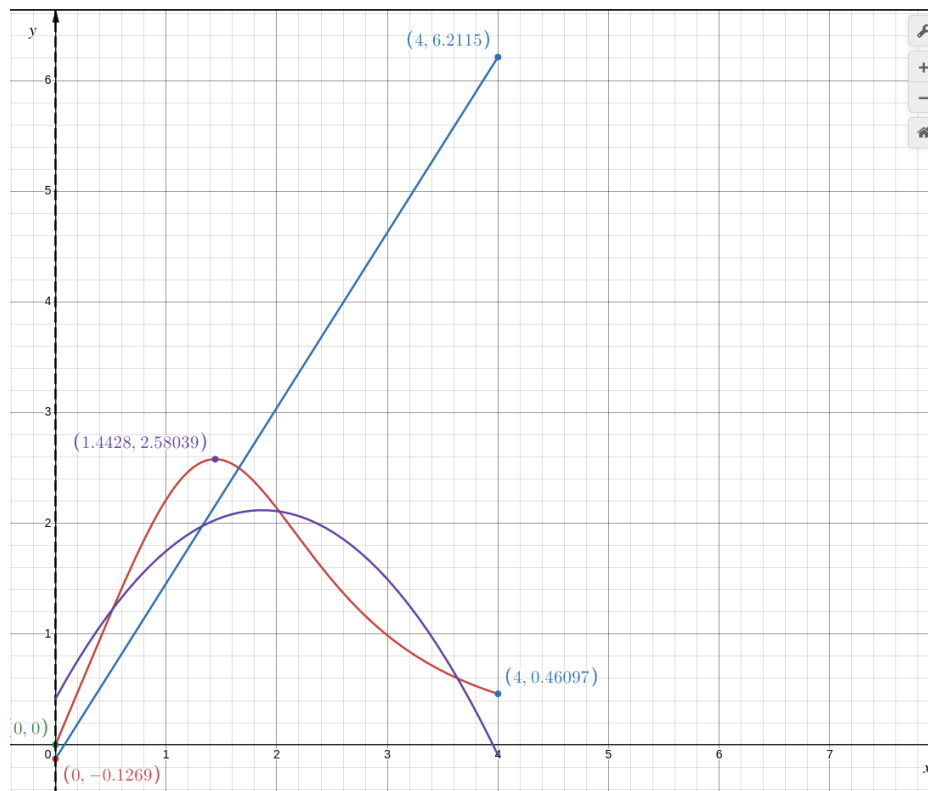
$$\boxed{\varphi(x) = 0.4154 + 1.8220x - 0.4872x^2}$$

Найдем среднеквадратическое отклонение:

i	0	1	2	3	4	5	6	7	8	9	10
x_i	0.0	0.4	0.8	1.2	1.6	2.0	2.4	2.8	3.2	3.6	4.0
y_i	0	0.951	1.849	2.468	2.537	2.138	1.611	1.166	0.842	0.617	0.461
$\varphi(x_i)$	0.415	1.066	1.561	1.900	2.083	2.111	1.982	1.697	1.257	0.660	-0.092
ε_i^2	0.173	0.013	0.083	0.322	0.206	0.001	0.138	0.282	0.172	0.002	0.306

$$\varepsilon_i = (\varphi(x_i) - y_i)$$

Сумма квадратов отклонений:



$$S = \sum_{i=0}^{10} \varepsilon_i^2 = 0.173 + 0.013 + 0.083 + 0.322 + 0.206 + 0.001 + 0.138 + 0.282 + 0.172 + 0.002 + 0.306 = 1.698$$

Среднеквадратичное отклонение:

$$\sigma = \sqrt{\frac{S}{n}} = \sqrt{\frac{\sum_{i=0}^{10} (\varphi(x_i) - y_i)^2}{11}} = \sqrt{\frac{1.698}{11}} = \sqrt{0.1544} \approx 0.393$$

$$\sigma_{\text{лин}} = 2.871, \quad \sigma_{\text{квадр}} = 0.393$$

Так как $\sigma_{\text{квадр}} < \sigma_{\text{лин}}$, **квадратичная аппроксимация является наилучшей** для данной табличной функции.

1.2 Программная часть

Исходный код доступен здесь – <https://github.com/SE-Itmo-Labs/nm-sem4-lab4>

Блок-схемы

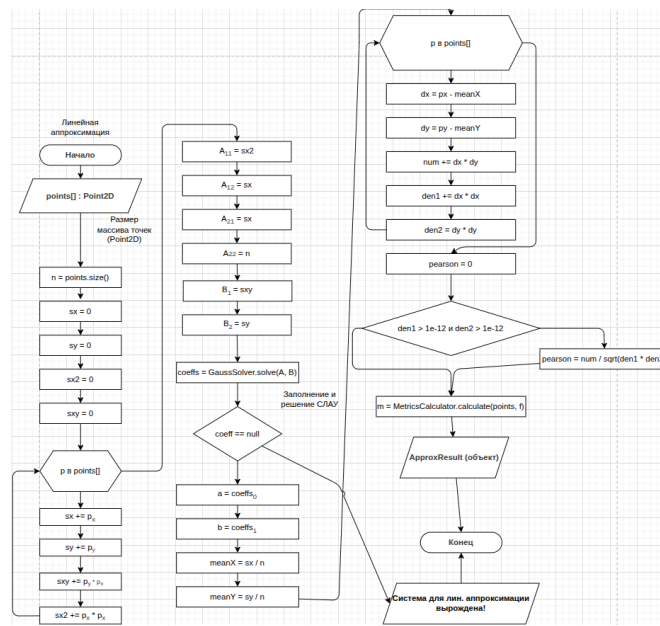


Рис. 1: Блок-схема для линейной аппроксимации

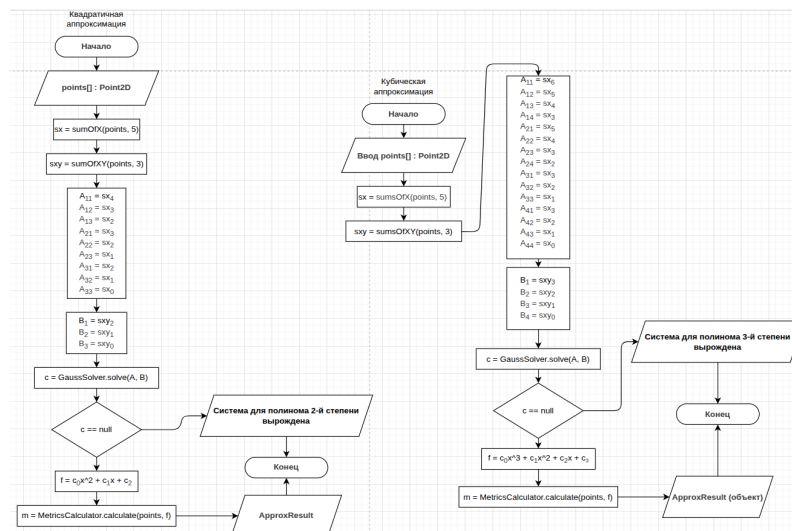


Рис. 2: Блок-схема для квадратичной и кубической аппроксимаций

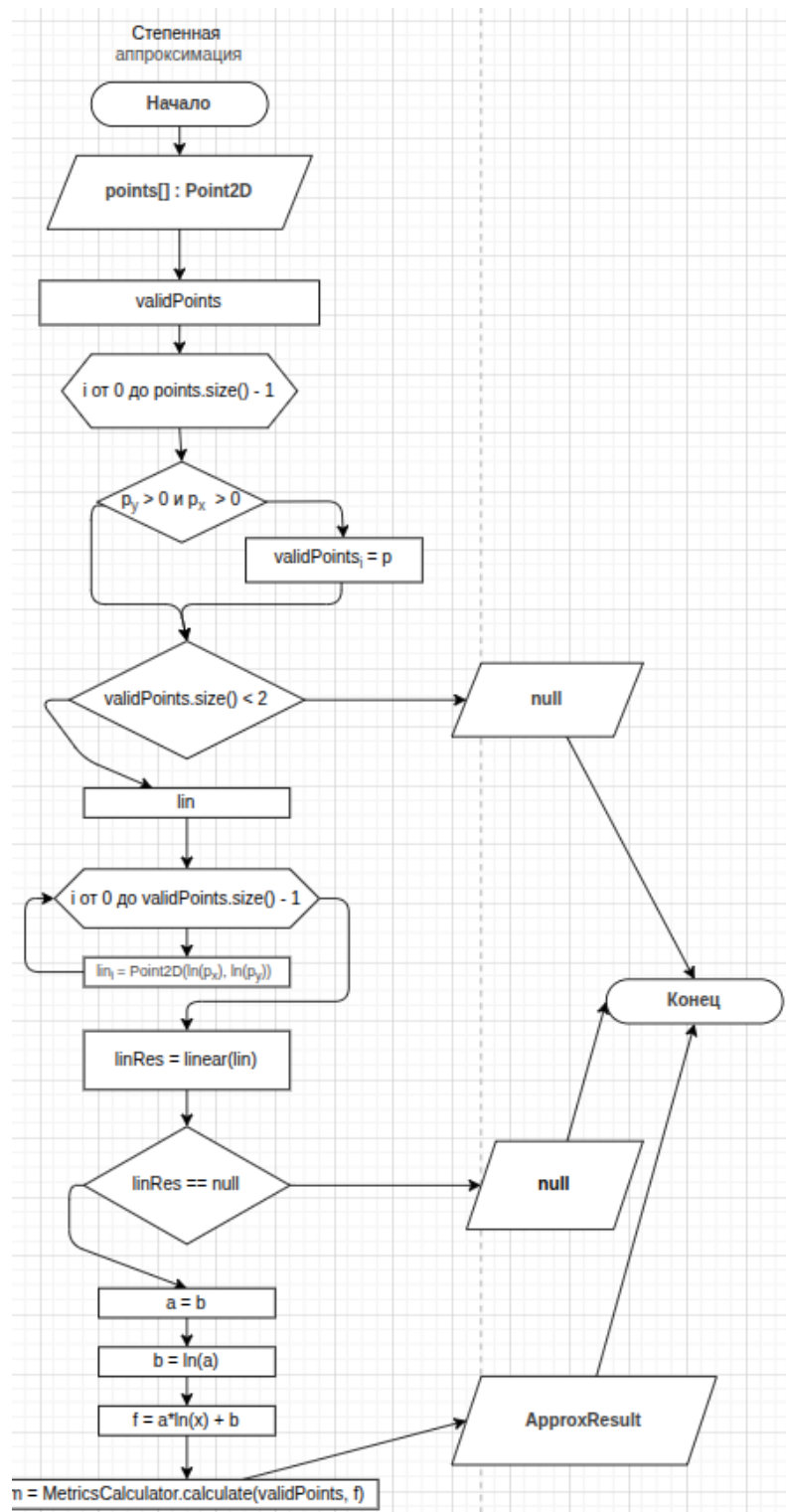


Рис. 3: Блок-схема для степенной аппроксимации

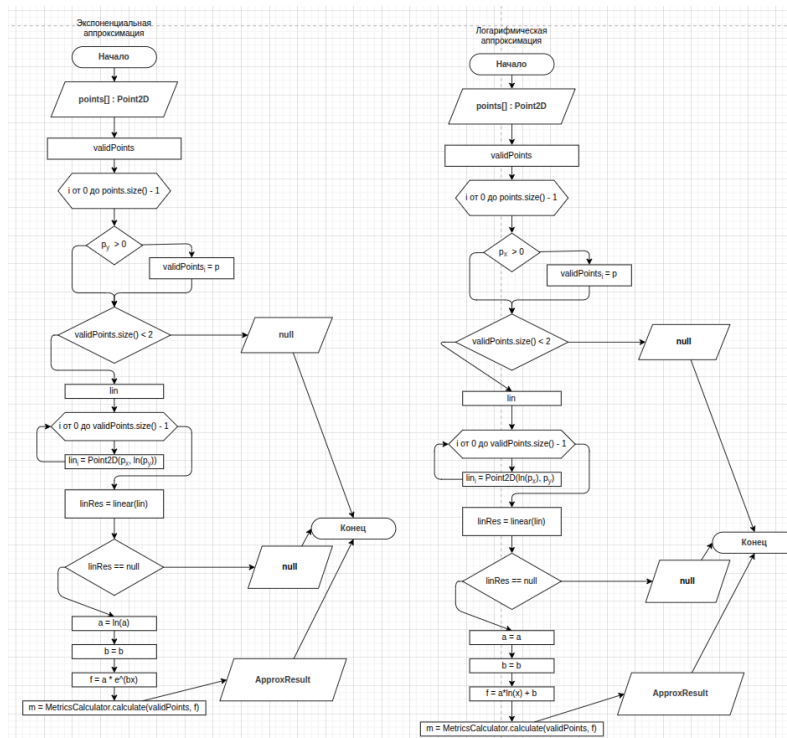


Рис. 4: Блок-схема для экспоненциальной и логарифмической аппроксимаций

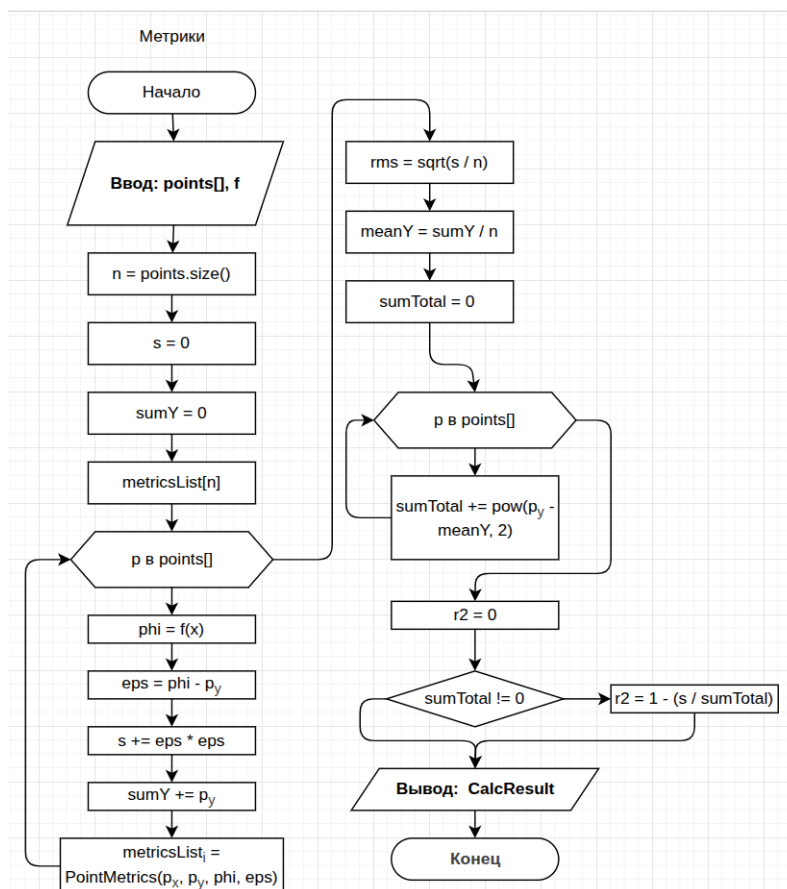


Рис. 5: Блок-схема для счета метрик

```

1 public ApproxResult linear(List<Point2D> points) {
2     double n = points.size();
3     double sx = points.stream().mapToDouble(p -> p.x).sum();
4     double sy = points.stream().mapToDouble(p -> p.y).sum();
5     double sx2 = points.stream().mapToDouble(p -> p.x * p.x).sum();
6     double sxy = points.stream().mapToDouble(p -> p.x * p.y).sum();
7
8     double[][] A = {{sx2, sx}, {sx, n}};
9     double[] B = {sxy, sy};
10    double[] coeffs = GaussSolver.solve(A, B);
11    if (coeffs == null) throw new IllegalArgumentException("Система
        для линейной аппроксимации вырождена ");
12
13    double a = coeffs[0];
14    double b = coeffs[1];
15    Function<Double, Double> f = x -> a * x + b;
16
17    // ПИРСОН
18    double meanX = sx / n;
19    double meanY = sy / n;
20
21    double num = points.stream().mapToDouble(p -> (p.x - meanX) * (p.y -
        meanY)).sum();
22    double den1 = points.stream().mapToDouble(p -> Math.pow(p.x - meanX, 2)
        ).sum();
23    double den2 = points.stream().mapToDouble(p -> Math.pow(p.y - meanY, 2)
        ).sum();
24    double pearson = (den1 > 1e-12 && den2 > 1e-12) ? num / Math.sqrt(den1
        * den2) : 0.0;
25
26    CalcResult m = MetricsCalculator.calculate(points, f);
27    return build(
28        "Линейная", String.format("y = %.4fx + %.4f", a, b), f, m,
        pearson, Map.of("a", a, "b", b)
29    );
30 }

```

```

1 public ApproxResult poly2(List<Point2D> points) {
2     double[] sx = sumsOfX(points, 5);
3     double[] sxy = sumsOfXY(points, 3);
4
5     double[][] A = {
6         {sx[4],
7             sx[3],
8             sx[2]}, {sx[3], sx[2], sx[1]}, {sx[2], sx[1], sx[0]}};
9     double[] B = {sxy[2], sxy[1], sxy[0]};
10    double[] c = GaussSolver.solve(A, B);
11
12    if (c == null)
13        throw new IllegalArgumentException("Система для полинома 2-
        степеней вырождена ");
14
15    Function<Double, Double> f = x -> c[0] * Math.pow(x, 2) + c[1] * x + c

```

```

16         [2];
17         CalcResult m = MetricsCalculator.calculate(points, f);
18
19         return build(
20             "Полином й2- степени",
21             String.format("y = %.4fx2 + %.4fx + %.4f", c[0], c[1], c[2]),
22             f,
23             m,
24             null,
25             Map.of("a2", c[0], "a1", c[1], "a0", c[2])
26         );
27     }

```

```

1 public ApproxResult poly3(List<Point2D> points) {
2     double[] sx = sumsOfX(points, 7);
3     double[] sxy = sumsOfXY(points, 4);
4
5     double[][] A = {{sx[6], sx[5], sx[4], sx[3]}, {sx[5], sx[4], sx[3], sx
6         [2]}, {sx[4], sx[3], sx[2], sx[1]}, {sx[3], sx[2], sx[1], sx[0]}};
7     double[] B = {sxy[3], sxy[2], sxy[1], sxy[0]};
8     double[] c = GaussSolver.solve(A, B);
9     if (c == null) throw new IllegalArgumentException("Система дляполиномай
10         3- степенивырождена ");
11
12     Function<Double, Double> f = x -> c[0] * Math.pow(x, 3) + c[1] * Math.
13         pow(x, 2) + c[2] * x + c[3];
14     CalcResult m = MetricsCalculator.calculate(points, f);
15     return build("Полином й3- степени", String.format("y = %.4fx3 + %.4fx2 +
16         %.4fx + %.4f", c[0], c[1], c[2], c[3]), f, m, null,
17         Map.of("a3", c[0], "a2", c[1], "a1", c[2], "a0", c[3]));
18 }

```

```

1 public ApproxResult powApprox(List<Point2D> points) {
2     // x > 0 И y > 0
3     List<Point2D> validPoints = points.stream().filter(p -> p.x > 0 && p.y
4         > 0).collect(Collectors.toList());
5     if (validPoints.size() < 2) return null;
6
7     List<Point2D> lin = validPoints.stream().map(p -> new Point2D(Math.log(
8         p.x), Math.log(p.y))).collect(Collectors.toList());
9     ApproxResult linRes = linear(lin);
10    if (linRes == null) return null;
11
12    double a = Math.exp(linRes.params.get("b"));
13    double b = linRes.params.get("a");
14
15    Function<Double, Double> f = x -> a * Math.pow(x, b);
16    CalcResult m = MetricsCalculator.calculate(validPoints, f);
17
18    return build("Степенная", String.format("y = %.4f * x%.4f", a, b), f, m
19        , null, Map.of("a", a, "b", b));
20 }

```

```

1 public ApproxResult logApprox(List<Point2D> points) {
2     // x > 0
3     List<Point2D> validPoints = points.stream().filter(p -> p.x > 0).
        collect(Collectors.toList());
4     if (validPoints.size() < 2) return null;
5
6     List<Point2D> lin = validPoints.stream().map(p -> new Point2D(Math.log(
        p.x), p.y)).collect(Collectors.toList());
7     ApproxResult linRes = linear(lin);
8     if (linRes == null) return null;
9
10    double a = linRes.params.get("a");
11    double b = linRes.params.get("b");
12    Function<Double, Double> f = x -> a * Math.log(x) + b;
13    CalcResult m = MetricsCalculator.calculate(validPoints, f);
14    return build("Логарифмическая", String.format("y = %.4f ln(x) + %.4f", a,
        b), f, m, null, Map.of("a", a, "b", b));
15 }

```

```

1 public ApproxResult expApprox(List<Point2D> points) {
2     // filter (y > 0)
3     List<Point2D> validPoints = points.stream().filter(p -> p.y > 0).
        collect(Collectors.toList());
4     if (validPoints.size() < 2) return null;
5
6     List<Point2D> lin = validPoints.stream().map(p -> new Point2D(p.x, Math
        .log(p.y))).collect(Collectors.toList());
7     ApproxResult linRes = linear(lin);
8     if (linRes == null) return null;
9
10    double a = Math.exp(linRes.params.get("b"));
11    double b = linRes.params.get("a");
12
13    Function<Double, Double> f = x -> a * Math.exp(b * x);
14    CalcResult m = MetricsCalculator.calculate(validPoints, f);
15
16    return build("Экспоненциальная", String.format("y = %.4f * e^(%.4fx)", a,
        b), f, m, null, Map.of("a", a, "b", b));
17 }

```

Примеры использования программы

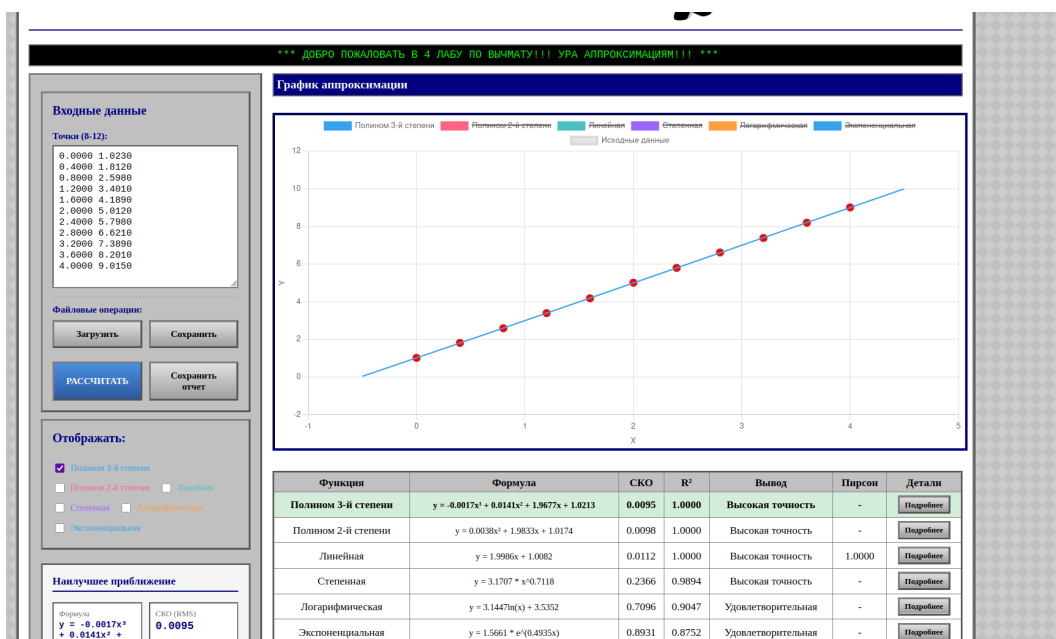


Рис. 6: Пример ввода значений для линейной аппроксимации

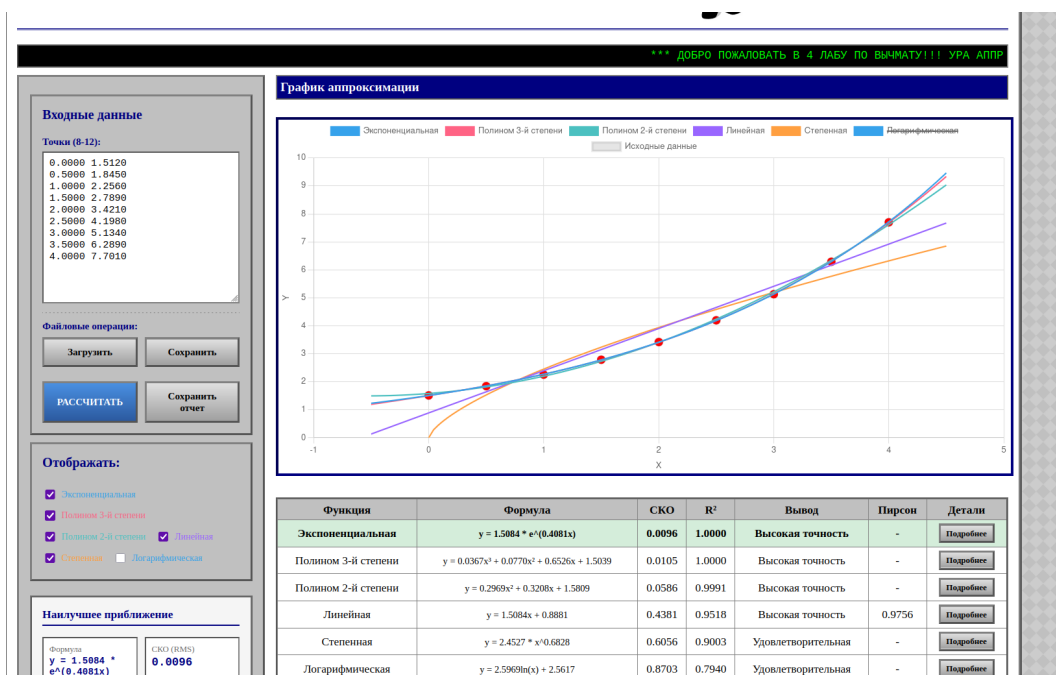


Рис. 7: Пример отображения с несколькими графиками

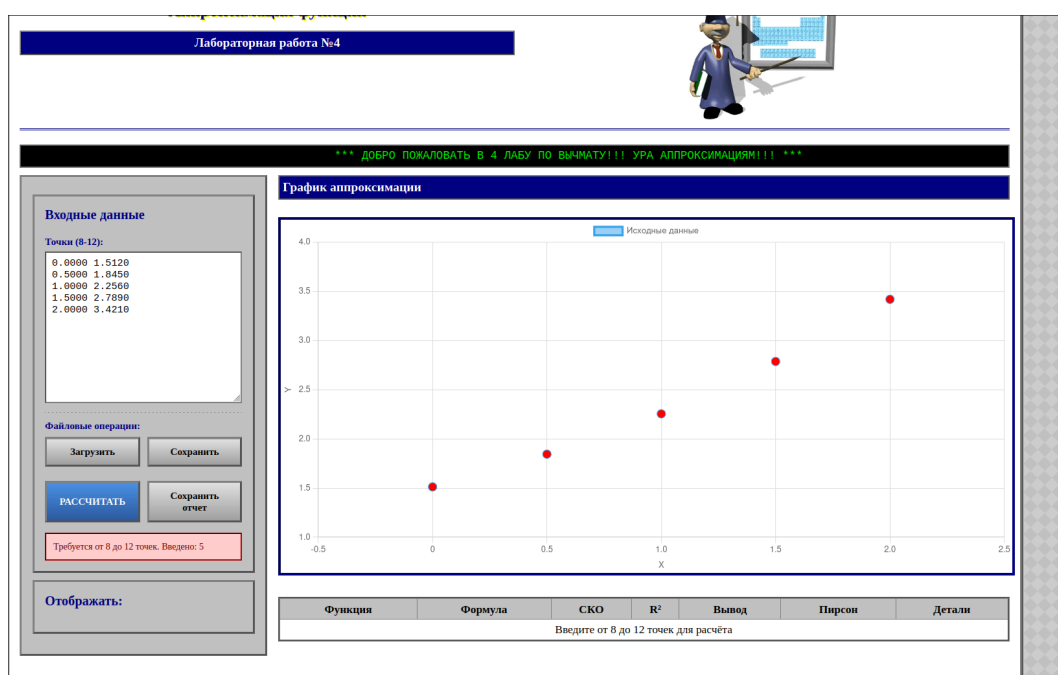


Рис. 8: Пример ввода недостаточного количества точек

2 Заключение

В ходе выполнения лабораторной работы я научился считать аппроксимации для различных функций (линейной, экспоненциальной, степенной, квадратичной, кубической, логарифмической), реализовал полноценный сервис для нахождения аппроксимирующей функции.